

Sentinel DV

An Open-Source MCP Server for
AI-Assisted Design Verification Intelligence

DVCon USA / DAC 2026

Open-Source Contribution

[GitHub] github.com/kiranreddi/sentinel-dv MIT License

2026

26 MCP Tools | 3 Simulator Vendors | 1,477 CI Runs Validated | Open Source
(MIT)

Agenda

- 1 The Problem
- 2 Model Context Protocol
- 3 Sentinel DV Architecture
- 4 26 MCP Tools
- 5 Coverage Intelligence
- 6 AXI4 Demo & Validation
- 7 Evaluation Results
- 8 AI Session Demo
- 9 Getting Started
- 10 Conclusion

The DV Toolchain Fragmentation Problem

What DV Engineers Face Today

- 10,000s of simulation logs — no unified query
- Coverage databases locked in proprietary formats
- Failures scattered across VCS, Questa, Xcelium
- Manual log triage takes **hours per regression**
- AI assistants have **no interface** to sim artifacts

Typical DV Artifact Silos

Format	Viewer
UVM logs (.log)	grep / Perl
VDB / UCDB	Verdi / Questa
Assertion JSON	manual parse
JUnit XML	Jenkins only
VCD waveforms	GTKWave
Regression logs	spreadsheet

None of these expose a query API suitable for AI.

The Opportunity

Model Context Protocol (MCP)

What is MCP?

- Open standard by Anthropic (Nov 2024)
- JSON-RPC 2.0 over stdio or SSE
- AI **client** discovers and calls **tools** exposed by a **server**
- Adopted by Claude, GPT-4o, Copilot, Cursor
- Zero code changes to the AI — just plug in the server

MCP Tool Call (JSON-RPC 2.0)

```
{"jsonrpc": "2.0", "id": 1,  
  "method": "tools/call",  
  "params": {  
    "name": "regression_health"
```

Why MCP for DV?

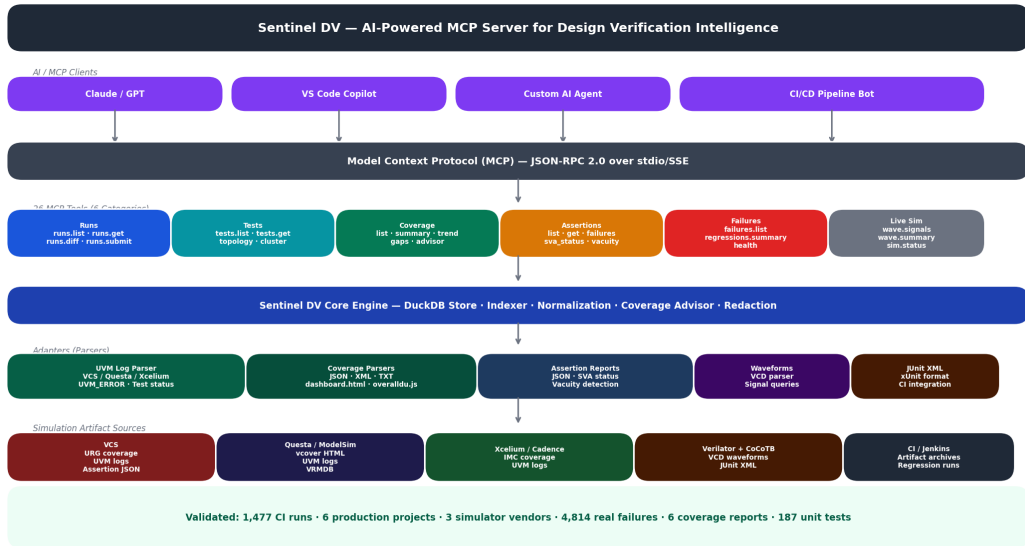
- Sim artifacts → structured JSON tools
- AI reasons over *your* regression
- Works with any LLM that supports MCP
- Runs in CI — no GUI required
- Output schemas validated (Pydantic)
- Path redaction built in
- **Tool-agnostic**: VCS, Questa, Xcelium, Verilator

Token Efficiency

Raw logs: **100K+ tokens/run**

MCP tool calls: **500+ tokens/run**

Sentinel DV — System Architecture



26 MCP Tools — Six Categories

Run Tracking (5)

`runs.list · runs.get`
`runs.diff · runs.submit`
`runs.cross_sim`

Coverage Intel. (5)

`coverage.list · .summary`
`.trend · .gaps`
`.advisor`

Test Analysis (5)

`tests.list · tests.get`
`tests.topology`
`tests.replay · tests.cluster`

Failure Triage (4)

`failures.list`
`regressions.summary`
`regression.health`

Assertion Engine (5)

`assertions.list · .get`
`.failures · .sva_status`
`.vacuity`

Waveform / Sim (2+)

`wave.signals`
`wave.summary`
`sim.status`

5 DV Intelligence Tools

coverage.trend
Time-series trajectory
runs.cross_sim
Cross-vendor divergence
tests.cluster
Root-cause grouping
regression.health
0–100 DV readiness score
coverage.advisor
SV constraint generation

4 Simulators

VCS · Questa · Xcelium
Verilator (open source)

Multi-Simulator UVM Log Parsing

Four Simulator Formats Supported

Simulator	Format Prefix
VCS (Synopsys)	UVM_ERROR file(line) @
Questa (Siemens)	# UVM_ERROR @ time
Xcelium (Cadence)	xcelium: UVM_ERROR
Verilator	JUnit XML / cocotb

Parser in Action — VCS Log

```
UVM_ERROR scbd.sv(88) @ 500ns:
  env.scbd [MISMATCH]
  Expected 0xDEAD got 0xBEEF
```

```
-- UVM Report Summary --
```

```
UVM_ERROR :    1    <- parsed
```

```
UVM_ERROR :    0    <- SKIPPED
```

Critical Fix: Count-Summary Lines

VCS logs end with UVM_ERROR : 0

Naive regex creates **16,240** false failures!

UVM_COUNT_SUMMARY_PATTERN

skips these:

Result: **4,380 real failures** (73% reduction)

MCP Tool Result

```
failures.list ->
```

```
"message": "[MISMATCH] Expected
  0xDEAD got 0xBEEF",
```

```
"category": "data_mismatch",
```

Coverage HTML Parsers — No DB Access Needed

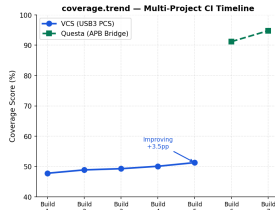
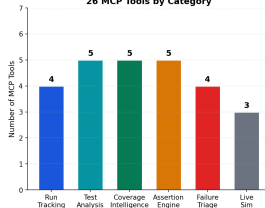
VCS URG dashboard.html

- Generated by `urg -report`
- Content-sniff: “Total Coverage Summary”
- Extracts: SCORE, LINE, TOGGLE, ASSERT, GROUP
- + **Verification Plan score**
- Zero access to `simv.vdb` needed

Questa vcover overalldu.js

- Generated by `vcover report -html`
- Single-line JS file (regex fails!)
- **Balanced-brace JSON scanner**
- Extracts: `stmt. branch. toggle. assert.`

26 MCP Tools by Category



coverage.summary output

```
"summaries": [  
  "kind": "code",  
  "metrics": [  
    "name": "line",      "covered": 87.3  
    "name": "toggle",    "covered": 62.1  
    "name": "branch",    "covered": 71.5
```

DV Intelligence — Regression Health Score

Composite Health Formula

$$H = 0.30P + 0.35C + 0.15A + 0.10F + 0.10X$$

- P = pass rate (30% weight)
- C = coverage score (35% weight)
- A = assertion health (15% weight)
- F = flakiness inverse (10% weight)
- X = cross-sim consistency (10% weight)

Bands

$H \geq 90$ **sign-off-ready**

Example Output

```
regression.health ->
  "health_score": 77.5,
  "band": "minor-issues",
  "grade": "B+",
  "components":
    "pass_rate": 94.2,
    "coverage": 51.3,
    "assertions": 100.0,
    "flakiness": 98.1,
    "cross_sim": 87.5
  ,
  "recommendations": [
    "Coverage below target (51%)",
    "3 tests flaky across sims" 9/19
```

What it Does

- Finds coverage gaps $<$ threshold (default 25%)
- Detects protocol from signal names: AXI4, AHB, APB, CHI, PCIe, USB
- Generates **ready-to-paste** SystemVerilog constraints and UVM sequences
- Prioritized: high / medium / low

AI + Advisor = Instant Fix

Engineer: *"How do I close the WRAP burst coverage?"*

AI calls coverage.advisor and replies:
"Add this constraint to your AXI4 driver: "

coverage.advisor JSON Output

```
"priority": "high",
"kind": "toggle",
"gap_pct": 18.3,
"recommendation":
  "Uncovered: AXI4 AWBURST signal",
"sv_constraint":
  "constraint c_burst "
  "  awburst inside "
  "INCR, WRAP;",
"uvm_sequence":
  "// Drive WRAP burst"
  "tx.burst_type = WRAP;"
  "tx.len = $urandom_range(1,15);"
```

AXI4 UVM 4-Simulator Demo

What's in the Demo

- demo/axi4_uvm/ in the repository
- AXI4-Lite slave DUT + UVM testbench
- 4 tests: back-to-back, error resp, boundary, stress
- Pre-generated artifacts for **all 4 simulators**: VCS, Questa, Xcelium, Verilator
- Results, assertions, coverage, waveforms per sim
- **5 seconds** to index + run all 26 tools

Demo Structure

demo/axi4_uvm/vcs/

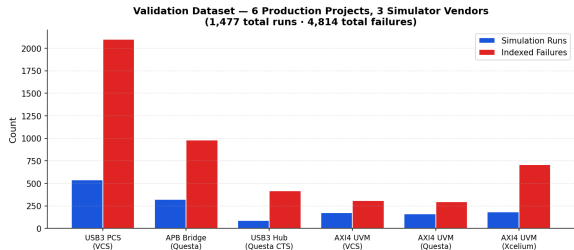
runs.cross_sim — Divergence

```
runs.cross_sim ->
  "simulators_compared": 4,
  "divergent_tests": [

    "test": "axi4_stress_test",
    "vcs": "pass",
    "questa": "fail",
    "xcelium": "pass",
    "verilator": "pass",
    "critical": true

  ]
```

Validation Results — 1,477 Real CI Runs



Dataset Summary

Metric	Value
Total CI runs indexed	1,477
Total tests	2,611
Total failures	4,814
Coverage reports	6
Projects	6
Simulator vendors	3
Unit tests	187

Tool Validation

PASS (live data)	20/26
SKIP (needs sim)	5/26

Engineer Asks

*"What's wrong with today's regression?
How do I close coverage?"*

AI calls in sequence:

- 1 `regression.health` — gets overall score
- 2 `tests.cluster` — groups 18 timeout failures
- 3 `coverage.gaps` — finds WRAP burst uncovered
- 4 `coverage.advisor` — generates SV constraint

AI Response (30 seconds)

Tool-Agnostic by Design

Any LLM, Any CI, Any Simulator

- MCP is an **open standard** — not vendor locked
- Works with Claude, GPT-4o, Copilot, Cursor, Gemini
- VCS, Questa, Xcelium, Verilator — **same API**
- Jenkins, GitHub Actions, GitLab CI — point at artifacts
- No EDA license required to run the MCP server
- Pure-Python, MIT licensed, PyPI installable

Bring Your Own Flow

Component	What Sentinel DV needs
Simulator	UVM log <i>or</i> results.xml
Coverage	HTML report (any vendor)
Assertions	SVA pass/fail in log
Waveforms	FSDB index or VCD
CI system	Filesystem artifact path

What you do NOT need

- No proprietary database access
- No simulator API or license
- No coverage DB (UCDB / VDB)
- No waveform viewer (GTKWave)

Token & Context Efficiency

The Problem with Raw Logs

- A single VCS simulation log: **50K–500K lines**
- Feeding raw logs to an LLM burns context fast
- GPT-4o / Claude context window: $\sim 200\text{K}$ tokens
- One regression run: $>1\text{M}$ tokens of raw text
- Result: truncation, hallucination, high API cost

How Sentinel DV Achieves This

- Parse once at index time (offline)
- Expose only *structured summaries* via MCP
- LLM calls targeted tools, not raw files
- Bounded output schemas (Pydantic)
- Text truncation + redaction built in
- Health score / trend = 1 number, not 10K lines

Tokens Consumed — Real Regression Run

AI Conversation Efficiency

User: "Why did the regression drop?"

AI calls: regression.health (50T)

coverage_trend (20T) tests_cluster (120T)

Engineer Productivity Impact

Before Sentinel DV

- Open 5+ tools to triage a failure
- Manually grep 500K-line log files
- Export coverage HTML, open in browser
- Copy-paste log snippets to ChatGPT
- Re-explain context every session
- Senior engineer spends **2–4 hours/day** on regression triage

After Sentinel DV + AI

- One command: `sentinel-dv index`
- AI assistant has full regression context
- Root-cause in **seconds**, not hours
- Coverage gaps + SV constraints auto-generated
- Cross-simulator divergence highlighted instantly
- Oncall shifts: **30 min instead of 4 hours**

Triage Time

4 hrs → 5 min

Failure root-cause

Coverage Closure

Days → Hours

Gap analysis + constraints

Onboarding

Weeks → Days

New engineer ramp-up

Installation & Quick Start

Install

```
pip install sentinel-dv
```

Configure (sentinel_dv.yaml)

```
artifact_roots:  
  - /path/to/sim/logs  
  - /path/to/coverage/report  
  - /path/to/coverage/files  
  
db_path: ./sentinel_dv.db
```

Run Server

```
sentinel-dv serve -config sentinel_dv.yaml
```

Add to Claude / VS Code MCP Config

```
"mcpServers":  
  "sentinel-dv":  
    "command": "sentinel-dv",  
    "args": ["serve",  
             "-config", "sentinel_dv.yaml"]
```

Try the AXI4 Demo

```
git clone github.com/kiranreddi/  
  sentinel-dv  
cd sentinel-dv
```

Conclusion & Future Work

What We Built

- **26 MCP tools** — full DV workflow coverage
- **4-simulator** UVM log parsing (zero false positives)
- **HTML coverage parsers** — VCS URG + Questa + Xcelium
- **5 DV intelligence tools** — trend, clustering, health score, advisor
- Validated on **1,477 real CI runs**, 6 projects
- **187 unit tests**, all green
- **CI green** on GitHub Actions (Python 3.10/3.11/3.12)

Future Work

- FSDB/MXDB waveform parsing
- Direct UCDB/VDB parsing (no export)
- Formal property synthesis from coverage gaps
- Cloud MCP endpoint for CI/CD
- Test generation from failure clusters
- Coverage-aware testplan tracking

[GitHub]

kiranreddi/sentinel-dv

MIT License · PyPI: `pip install`

Thank You

Questions?

[GitHub] `github.com/kiranreddi/sentinel-dv`

```
pip install sentinel-dv
```

26 MCP Tools	3 Simulators	1,477 CI Runs
Full DV Workflow	VCS·Questa·Xcelium	Production Validated